

Drone Firmware Engineer Roadmap

Starting point: from-scratch C flight controller sim (quaternion physics, RK4, cascaded PID, motor mixer) + ArduPilot SITL flights, both running. Learning style: implement first, understand the math after.

This roadmap is organized in phases. Each phase builds on working code from the last one – you don't need to "finish learning" a topic before moving on, just get something running, then loop back and deepen it.

Phase 0 – Where you are now

- [x] Rigid-body quadrotor physics simulator (RK4 integration)
- [x] Quaternion attitude representation
- [x] Cascaded PID (rate → attitude → altitude hold)
- [x] Motor mixer
- [x] ArduPilot SITL flying, basic parameter/code modification exercises

You're at the boundary between "control theory works in an idealized sim" and "control theory works with realistic sensors." That boundary is Phase 1.

Phase 1 – State Estimation (next up)

Goal: stop feeding the controller perfect simulated truth; feed it what a real IMU would actually give you.

1. **Noisy sensor model** – add gyro/accel noise, bias drift, and fixed sample rate to your simulator (you already have this queued).
2. **Complementary filter** – fuse gyro (fast, drifts) with accel (slow, noisy, but bias-free) to estimate attitude. This is the "hello world" of sensor fusion.
3. **Understand why it works** – after it runs, dig into the math: it's a simple frequency-domain trick (high-pass the gyro, low-pass the accel).
4. **Kalman filter** – once complementary filter clicks, implement a basic KF for a 1D case (e.g., altitude from baro + accel), then generalize.
5. **Extended Kalman Filter (EKF)** – the real target. This is what ArduPilot (AP_NavEKF3) and PX4 use for full state estimation (attitude, position, velocity, sensor biases) from IMU + GPS + baro + mag.
6. **Sensor-specific quirks** to model/handle: gyro bias drift, accel vibration noise, magnetometer hard/soft iron distortion, GPS/IMU rate mismatch (GPS ~5-10Hz vs IMU ~400-1000Hz), sensor calibration routines (gyro bias at boot, accel/mag ellipsoid fit).

Milestone: your sim flies correctly using *estimated* state instead of ground-truth state, and stays stable under injected sensor noise.

Phase 2 – Control Theory, Deeper

You have working cascaded PID – now make it robust.

- Integrator anti-windup (clamping, back-calculation)
- Derivative-on-measurement vs derivative-on-error (avoids "derivative kick" on setpoint changes)
- Feedforward terms (angular acceleration feedforward, especially useful for racing/acro tunes)
- Gain scheduling (different gains at different speeds/battery voltages)
- Cascade bandwidth separation – rule of thumb: each outer loop should run ~5-10x slower in bandwidth than the loop inside it
- Compare against ArduPilot's `AC_AttitudeControl` and `AC_PID` source once yours is solid – great way to sanity-check your design against a flight-proven one

Milestone: controller stays stable across a wider disturbance/noise envelope, no windup-related oscillation, clean step response.

Phase 3 – Real-Time Execution & Scheduling

This is the conceptual jump from "sim on a Mac" to "firmware on a microcontroller."

- Fixed-rate task scheduling (e.g., 400Hz control loop, 50Hz telemetry, 10Hz logging – all running off one timer)
- Interrupt-driven sensor reads (IMU data-ready pin triggers a read, not polling)
- Jitter and its effect on control stability – a loop that's occasionally late is often worse than one that's consistently a bit slower

- RTOS concepts (FreeRTOS / ChibiOS / NuttX – what ArduPilot and PX4 actually run on) vs a simple bare-metal super-loop
- Where your current simulator's "just call update() every iteration" model breaks down once timing becomes non-negotiable

Milestone: you understand (even without hardware yet) how your control loop's timing assumptions from Phase 0-2 would need to change on real firmware.

Phase 4 – Hardware Peripheral Interfacing

Once Pi Zero (or another dev board) arrives, this is where sim meets metal.

- SPI (preferred for high-rate IMU reads) / I2C (secondary sensors, slower)
- UART for GPS and telemetry radios
- PWM / OneShot125 / **DSHOT** for ESC output – DShot is digital with checksums, much more reliable than analog PWM, worth learning even if you start with plain PWM
- DMA – offloading peripheral transfers so the CPU isn't blocked on sensor/actuator I/O
- Timer configuration for PWM generation, ADC for battery voltage/current sensing

Milestone: read real sensor data over SPI/I2C on hardware and drive a real (or bench-tested) ESC output.

Phase 5 – Motor Mixing & Actuator Handling, Advanced

You have basic mixing – extend it toward what real firmware does.

- Mixing matrices for multiple frame types (X, +, hex, octo, custom)
- Motor saturation handling – when one motor maxes out, preserve control authority on the *other* axes instead of just clipping (this is a real design decision in ArduPilot/Betaflight mixers)
- ESC protocol selection and telemetry (DSHOT can report RPM back)

Phase 6 – Failsafe & Safety Architecture

Often skipped by hobby projects, but it's most of what separates "flies in sim" from "safe to fly for real." Worth learning conceptually even before hardware.

- Watchdog timers
- Sensor health monitoring (detect a dead/stuck/out-of-range IMU and react)
- Failsafe state machines: RC signal loss, GPS loss, battery critical, EKF divergence
- Pre-arm checks (the boring-but-critical checklist real flight stacks run before allowing motor start)

Phase 7 – Communication & Ground Control

You're already touching this via SITL/MAVProxy.

- MAVLink message structure, telemetry streams vs command streams
- Parameter protocol (how `mav.parm`-style persistence actually works under the hood)
- Reading ArduPilot's actual estimator (`AP_NavEKF3`) and attitude controller (`AC_AttitudeControl`) source once your own versions are working – comparing your implementation against a flight-proven one is one of the best ways to deepen understanding

Phase 8 – Coordinate Frames & Math (ongoing, not sequential)

You've got quaternions. Layer in as needed:

- NED vs ENU frame conventions (ArduPilot/PX4 both care about this, and it trips people up)
- Body frame vs earth frame rotations
- Gimbal lock awareness (you're avoiding it via quaternions, but understanding *why* Euler angles fail helps when reading other people's code/docs that still use them)

Suggested sequencing for you specifically

Given your current state and style:

1. **Now:** sensor noise model + complementary filter (Phase 1, steps 1-3)
2. **Next:** basic Kalman filter on a simple 1D case, then EKF (Phase 1, steps 4-5)
3. **In parallel:** anti-windup + derivative-on-measurement fixes to your existing PID (Phase 2 – small, high-value changes to code you already have)
4. **Once EKF is solid:** revisit ArduPilot SITL source for the equivalent modules – you'll actually understand what you're reading now
5. **When Pi Zero arrives:** Phase 3 (scheduling/timing) and Phase 4 (peripherals) become concrete instead of theoretical

Everything after that (Phases 5-8) can be layered in opportunistically as your stack matures – they're less about new hard concepts and more about robustness and breadth.